

Sprawozdanie: Tryby pracy szyfrów blokowych

Jakub Biernat 160248

1. Kod aplikacji:

```
from secrets import choice
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
import time
import matplotlib.pyplot as plt

MODES = {
    "ECB": AES.MODE_ECB,
    "CBC": AES.MODE_CBC,
    "CFB": AES.MODE_CFB,
    "OFB": AES.MODE_OFB,
    "CTR": AES.MODE_CTR
}

def add_padding(text):
    text = text.encode()
    return text + b' ' * (16 - len(text) % 16)

def generate_cipher(mode, key):
    if mode == AES.MODE_ECB:
        cipher = AES.new(key, mode)
    elif mode == AES.MODE_CTR:
        return AES.new(key, mode, nonce=get_random_bytes(8))
    else:
        iv = get_random_bytes(16)
        cipher = AES.new(key, mode, iv=iv)

    return cipher

def generate_ciphertext(cipher, text):
    return cipher.encrypt(text)

def generate_decipher(mode, key, cipher):
    if mode == AES.MODE_ECB:
        decipher = AES.new(key, mode)
    elif mode == AES.MODE_CTR:
        decipher = AES.new(key, mode, nonce=cipher.nonce)
    else:
        decipher = AES.new(key, mode, iv=cipher.iv)

    return decipher

def generate_decrypted(decipher, ciphertext):
    return decipher.decrypt(ciphertext)
```

```

def compare_modes(texts):
    key = get_random_bytes(16)
    input_bits = []
    enc_times = {name: [] for name in MODES.keys()}
    dec_times = {name: [] for name in MODES.keys()}

    for text in texts:
        text = add_padding(text)
        input_bits.append(len(text) * 8)
        for mode_name, mode in MODES.items():
            cipher = generate_cipher(mode, key)
            start_enc = time.perf_counter()
            ciphertext = generate_ciphertext(cipher, text)
            stop_enc = time.perf_counter()
            enc_times[mode_name].append(stop_enc - start_enc)

            decipher = generate_decipher(mode, key, ciphertext)
            start_dec = time.perf_counter()
            decrypted = generate_decrypted(decipher, ciphertext)
            stop_dec = time.perf_counter()
            dec_times[mode_name].append(stop_dec - start_dec)

    plt.figure(figsize=(12, 6))

    ax1 = plt.subplot(1, 2, 1)
    for mode in MODES.keys():
        ax1.plot(input_bits, enc_times[mode], label=mode)

    ax1.set_xlabel("Długość danych wejściowych (bity)")
    ax1.set_ylabel("Czas szyfrowania (sekundy)")
    ax1.set_title("Czas szyfrowania AES w zależności od długości danych")
    ax1.legend()
    ax1.grid()
    ax1.ticklabel_format(style='plain', axis='x')

    ax2 = plt.subplot(1, 2, 2)
    for mode in MODES.keys():
        ax2.plot(input_bits, dec_times[mode], label=mode)

    ax2.set_xlabel("Długość danych wejściowych (bity)")
    ax2.set_ylabel("Czas deszyfrowania (sekundy)")
    ax2.set_title("Czas deszyfrowania AES w zależności od długości danych")
    ax2.legend()
    ax2.grid()
    ax2.ticklabel_format(style='plain', axis='x')

    plt.tight_layout()
    plt.savefig("Wykresy.png", dpi=300)
    plt.show()

    return

```

```

def flip_bit(text):
    byte_arr = bytearray(text)
    byte_index = choice(range(len(byte_arr)))
    bit_index = choice(range(8))
    byte_arr[byte_index] ^= (1 << bit_index)
    return bytes(byte_arr)

def byte_difference(originaltext, decrypted):
    length = min(len(originaltext), len(decrypted))
    return sum(1 for i in range(length) if originaltext[i] != decrypted[i])

def test_modes_error_propagation():
    text = "Litwo! Ojczyzna moja! ty jesteś jak zdrowie. Ile cie trzeba cenic,
           ten tylko sie dowie. Kto cie stracił."
    print("Test propagacji błędu")
    print(f"Oryginalny tekst: {text}")
    key = get_random_bytes(16)

    padded_text = add_padding(text)
    for mode_name, mode in MODES.items():
        cipher = generate_cipher(mode, key)
        ciphertext = generate_ciphertext(cipher, padded_text)

        corrupted = flip_bit(ciphertext)

        decipher = generate_decipher(mode, key, cipher)
        decrypted = generate_decrypted(decipher, corrupted)

        byte_diff = byte_difference(padded_text, decrypted)
        decrypted = decrypted.decode('utf-8', errors='ignore')
        print(f"\nPropagacja błędu {mode_name}")
        print(f"Odszyfrowany tekst: {decrypted}")
        if byte_diff == 1:
            print("1 bajt został zmieniony, brak propagacji na inne bajty")
        elif byte_diff <= 16:
            print(f"1 blok uszkodzony, zmiana uległa {byte_diff} bajtów")
        else:
            print(f"Propagacja błędu, zmiana uległa {byte_diff} bajtów")

def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

```

```

def encrypt_cbc_ecb(text, key, iv):
    cipher_ecb = generate_cipher(AES.MODE_ECB, key)

    text = add_padding(text)
    ciphertext = b""
    prev_block = iv

    for i in range(0, len(text), 16):
        block = text[i : i + 16]
        block = xor(block, prev_block)
        encrypted = cipher_ecb.encrypt(block)
        ciphertext += encrypted
        prev_block = encrypted

    return iv + ciphertext

def decrypt_cbc_ecb(key, ciphertext):
    decipher_ecb = generate_cipher(AES.MODE_ECB, key)

    iv = ciphertext[:16]
    ciphertext = ciphertext[16:]
    decrypted = b""
    prev_block = iv

    for i in range(0, len(ciphertext), 16):
        block = ciphertext[i : i + 16]
        dec = decipher_ecb.decrypt(block)
        decrypted += xor(dec, prev_block)
        prev_block = block

    return decrypted

def main():
    #Podpunkt 1
    texts = ["a" * i for i in range(100000, 1000000, 100000)]
    compare_modes(texts)

    #Podpunkt 2
    test_modes_error_propagation()

    #Podpunkt 3
    text = "Ala ma kota."
    key = get_random_bytes(16)
    iv = get_random_bytes(16)
    ciphertext = encrypt_cbc_ecb(text, key, iv)
    decrypted = decrypt_cbc_ecb(key, ciphertext)
    decrypted = decrypted.decode('utf-8', errors='ignore')
    print("\nTest implementacji CBC przy uzyciu ECB")
    print(f"Oryginalny tekst: {text}")
    print(f"Odszyfrowany tekst: {decrypted}")

main()

```

2. Omówienie sposobu implementacji

Wybrany język programowania: Python

Użyte biblioteki:

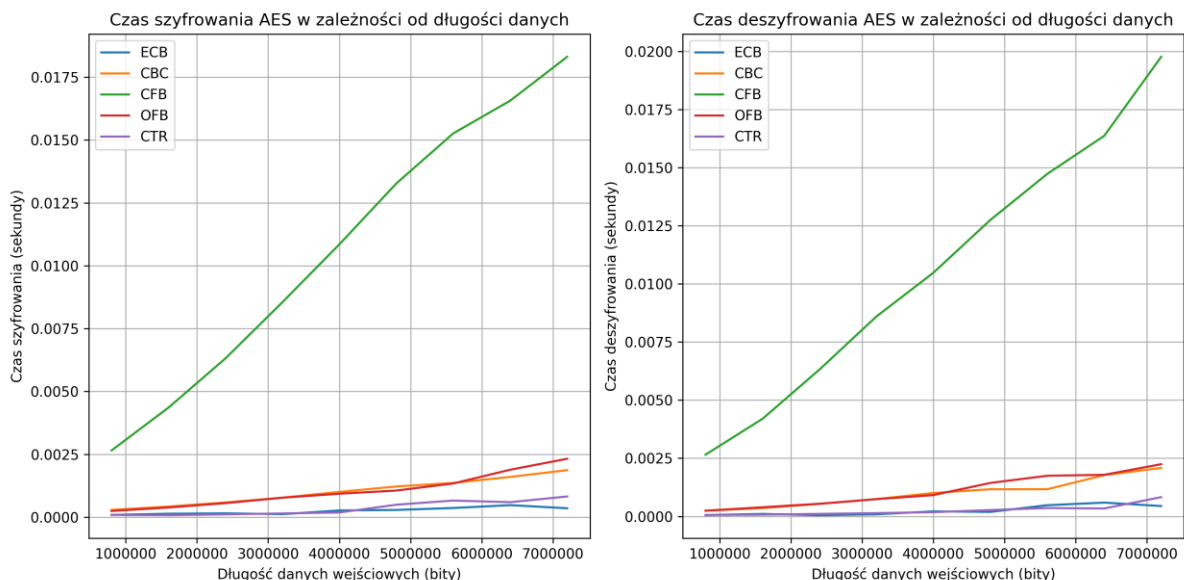
- Crypto- wykorzystanie trybów pracy AES
- time- obliczanie czasu działania trybów
- matplotlib- wizualizacja wyników
- secrets- funkcja losująca do generowania losowych danych

Główne elementy implementacji:

- add_padding()- dopasowuje długość tekstu do wielokrotności 16 bajtów
- generate_cipher()- tworzy obiekt szyfrujący AES w podanym trybie
- generate_ciphertext()- szyfruje podany tekst
- generate_decipher()- sprawdza efekt lawinowy funkcji
- generate_decrypted()- zliczanie różnicy bitów
- compare_modes()- porównuje czasy szyfrowania i deszyfrowania różnych trybów AES
- flip_bit()- losowo zmienia jeden bit w losowym bajcie
- byte_difference()- liczy ile bajtów różni się między oryginalnym a odszyfrowanym tekstem
- test_modes_error_propagation()- testuje jak błąd w jednym bicie szyfrogramu wpływa na odszyfrowany tekst w różnych trybach AES
- xor()- wykonuje operację XOR na dwóch ciągach bajtów
- encrypt_cbc_ecb()- ręcznie implementuje szyfrowanie CBC przy użyciu ECB
- decrypt_cbc_ecb()- ręcznie odszyfrowuje dane zaszyfrowane CBC przy użyciu ECB

3. Zestawienie uzyskanych wyników wraz ze stosownymi wnioskami

- Eksperyment z porównaniem czasów działania różnych trybów AES w zależności od długości danych wejściowych:



Wnioski: Tryb CFB jest zdecydowanie najwolniejszym trybem, oraz czas jego wykonania najszybciej rośnie, wykresy szyfrowania i deszyfrowania są podobne.

- Eksperyment z badaniem testowaniem propagacji błędów:

Test propagacji błędów

Oryginalny tekst: Litwo! Ojczyzno moja! ty jesteś jak zdrowie. Ile cie trzeba cenic, ten tylko sie dowie. Kto cie stracił.

Propagacja błędów ECB

Odszyfrowany tekst: Litwo! Ojczyzno moja! ty jesteś 7;1l" cie trzeba cenic, ten tylko sie dowie. Kto cie stracił.

1 blok uszkodzony, zmianie uległo 16 bajtów

Propagacja błędów CBC

sn IUmoja! ty kestes jak zdrowie. Ile cie trzeba cenic, ten tylko sie dowie. Kto cie stracił.

Propagacja błędów, zmianie uległo 17 bajtów

Propagacja błędów CFB

Odszyfrowany tekst: Litwo! Ojczyzno moja! ty jesteś jak zdrowie. Ile cie trzeba cenic, ten tylko sie dowie. Kto cie stracił. ZK

1 blok uszkodzony, zmianie uległo 5 bajtów

Propagacja błędu OFB

*Odszyfrowany tekst: Litwo! Ojczyzna moja! ty jesteś jak zdrowie. Ile cie trzeba
cenic, ten tylko sie dowie. Kto cie stracił.*

1 bajt został zmieniony, brak propagacji na inne bajty

Propagacja błędu CTR

*Odszyfrowany tekst: Litwo! Ojczyzna moja! ty jeStes jak zdrowie. Ile cie trzeba
cenic, ten tylko sie dowie. Kto cie stracił.*

1 bajt został zmieniony, brak propagacji na inne bajty

- Eksperyment z implementacją CBC przy użyciu ECB:

Test implementacji CBC przy użyciu ECB

Oryginalny tekst: Ala ma kota.

Odszyfrowany tekst: Ala ma kota.

Eksperyment się powiódł

- Eksperyment z badaniem kolizji:

Kolizja na pierwszych 12 bitach (SHA-3-512) znaleziona po 87 próbach:

Tekst 1: kSgojXWYu8M0hzdI

Tekst 2: EG1BbyZfjCKfGM4b

Wspólne 12 bitów: 101110100010

Hash 1:

*ba2c8a30757fbdb053ac6b58b8379e950c0fa7d7e21d7b3bdf5387a4613aec5e0
76ea991da668efc5bf24b57fc82448e3748b32ed602f21dbbaafb355a770d8d*

Hash 2:

*ba205e0fccd21f4f1aacb62e306a320868c06884f7c7b480f43f76ce6df8574c648
4f597f0b7d06c24c1d4f93a095ff508f88de1fd81f06795286777f1743b41*

- Eksperyment z badaniem efektu lawinowego:

Avalanche test (SHA-3-512):

Po 200 próbach, średni procent zmienionych bitów po zmianie jednego bitu na wejściu: 0.498779296875

Eksperyment się powiódł